

German International University
Faculty of Informatics and Computer Science
Dr. John Zaki
Amir Haytham
Hassan Abdelqader
Roba Askar
Nada Farid

Software Engineering, Spring 2026
GIU Nexus — AI-Powered Career & Talent Platform
Project - Milestone 2 - Backend Development

1 Overview

Milestone 1 gave your platform a skeleton — a data model and a plan. Milestone 2 is where it comes alive. You will build the complete **Node.js/Express** backend that powers GIU Nexus: user authentication with JWT, role-based access control, a full RESTful API, and — the part that makes this more than a plain CRUD app — three Hugging Face AI integrations that deliver real intelligence to your users.

By the end of this milestone, your backend should stand entirely on its own. A recruiter should be able to register, get approved, post a job, and have the system automatically classify it — all without touching the frontend. A job seeker should be able to extract skills from their bio and receive AI-ranked job recommendations. Test everything through **Postman** or **Thunder Client**; you should not need a browser at all.

Note on admin accounts: There is no `/register` path for the `admin` role. Admin users must be created directly in the database (e.g., via a seed script or MongoDB Compass). This is intentional — it mirrors real-world practice where super-user accounts are never self-provisioned.

2 Project Setup

Dependencies

Initialise your project with `npm init -y` and install the packages below. Add `start: "node server.js"` and `dev: "nodemon server.js"` to the `scripts` section of your `package.json`.

Package	Purpose
<code>express</code>	HTTP server and routing
<code>mongoose</code>	MongoDB object modelling
<code>dotenv</code>	Environment variable loading
<code>bcryptjs</code>	Password hashing
<code>jsonwebtoken</code>	JWT generation and verification
<code>@huggingface/inference</code>	Hugging Face Inference API client
<code>nodemailer</code>	Password reset emails
<code>cors</code>	Cross-Origin Resource Sharing
<code>nodemon</code> (dev)	Auto-restart during development

Recommended Project Structure

Keep your code organised from day one. Here is the folder layout your backend should follow:

```

1 server/
2   config/
3     db.js           <- MongoDB connection helper
4   models/
5     User.js
6     JobPost.js
7     Application.js
8   controllers/
9     authController.js
10    userController.js
11    jobController.js
12    applicationController.js
13    profileController.js
14  routes/
15    authRoutes.js
16    userRoutes.js
17    jobRoutes.js
18    applicationRoutes.js
19    profileRoutes.js
20  middleware/
21    auth.js          <- protect + authorize
22    errorHandler.js
23  services/
24    hfService.js     <- HuggingFace client singleton
25    emailService.js  <- Nodemailer helper
26  server.js
27  .env
28  .env.example
29  .gitignore

```

Environment Variables

Create a `.env` file in your server root and **never commit it** — add it to `.gitignore` immediately. Provide a `.env.example` with placeholder values so teammates know what keys are required.

Variable	Example value	Used by
PORT	5000	server.js
MONGO_URI	mongodb+srv://...	config/db.js
JWT_SECRET	a-long-random-string	auth middleware
JWT_EXPIRE	7d	JWT signing
HF_TOKEN	hf_xxxxxxxxxxxxx	hfService.js
EMAIL_HOST	smtp.gmail.com	emailService.js
EMAIL_PORT	587	emailService.js
EMAIL_USER	you@gmail.com	emailService.js
EMAIL_PASS	your-app-password	emailService.js

3 Authentication

Authentication is the backbone of your API — almost every route needs to know *who* is making the request and *what* they are allowed to do. You will implement this with **JSON Web Tokens (JWT)** for session management and **bcryptjs** for password hashing, the same combination used in production systems at scale.

Registration & Login Flow

- **Registration:** Accept name, email, password, and role (jobSeeker or recruiter). Hash the password with `bcryptjs` before saving. Recruiters are created with `status: "pending"` — they cannot post jobs until an admin explicitly approves them.

- **Login:** Verify the email/password pair, then sign a JWT containing `_id` and `role`. Return the token in the response body so the client can store it.
- **Logout:** Stateless — instruct the client to discard the token. Token blacklisting is available as a bonus (see Section 7).
- **Forgot Password:** Generate a cryptographically random reset token, hash it, store it on the user document with a 10-minute expiry, and send the raw token in a reset link via Nodemailer. The `PATCH /reset-password/:token` endpoint hashes the incoming token, looks it up, verifies expiry, and updates the password.

Middleware

These two functions will be imported by almost every route file:

- `protect` — Reads the `Authorization: Bearer <token>` header, verifies the JWT with your `JWT_SECRET`, queries the full user from the database, and attaches it to `req.user`. Returns 401 immediately if the token is missing, malformed, or expired.
- `authorize(...roles)` — A factory function that returns a middleware. It checks whether `req.user.role` is in the allowed list and returns 403 if not. Always chain it *after* `protect`. Example usage: `router.post('/', protect, authorize('recruiter'), createJob)`.

Auth Endpoint Reference

POST /api/v1/auth/register *Public. Creates a new user account.*

Field	Type	Required	Notes
<code>name</code>	String	Yes	Display name
<code>email</code>	String	Yes	Must be unique
<code>password</code>	String	Yes	Min 6 characters
<code>role</code>	String	Yes	"jobSeeker" or "recruiter"

```

1 // Response 201
2 {
3   "success": true,
4   "token": "<signed JWT>",
5   "user": { "_id": "...", "name": "Sara Ahmed", "email": "sara@example.com",
6             "role": "jobSeeker", "status": "approved" }
7 }
8 // Note: recruiters are returned with status "pending"
9 // Error 400: { "success": false, "message": "Email already in use" }
```

POST /api/v1/auth/login *Public. Returns a JWT for a valid email/password combination.*

```

1 // Request body
2 { "email": "sara@example.com", "password": "secret123" }
3 // Response 200
4 {
5   "success": true,
6   "token": "<signed JWT>",
7   "user": { "_id": "...", "name": "Sara Ahmed", "email": "sara@example.com",
8             "role": "jobSeeker", "status": "approved",
9             "profilePicture": "", "skills": ["React", "Node.js"] }
10 }
11 // Error 401: { "success": false, "message": "Invalid email or password" }
```

POST /api/v1/auth/logout *Private. Stateless logout — instruct the client to discard the token.*

```
1 // Header: Authorization: Bearer <token>
2 // Response 200
3 { "success": true, "message": "Logged out successfully" }
```

POST /api/v1/auth/forgot-password *Public. Sends a password reset link to the given email address. Always responds with 200 to avoid email enumeration.*

```
1 // Request body
2 { "email": "sara@example.com" }
3 // Response 200
4 { "success": true, "message": "Password reset email sent" }
```

PATCH /api/v1/auth/reset-password/:token *Public. Resets the password using the token received via email.*

Field	Type	Required	Notes
password	String	Yes	New password, min 6 characters

```
1 // Response 200
2 { "success": true, "token": "<new JWT>",
3   "user": { "_id": "...", "name": "...", "role": "..." } }
4 // Error 400: { "success": false, "message": "Token is invalid or has expired" }
```

4 API Routes

The table below is a quick reference of all endpoints. Full request/response documentation for each group follows in the subsections.

Route	Method	Access
/api/v1/auth/register	POST	Public
/api/v1/auth/login	POST	Public
/api/v1/auth/logout	POST	Private
/api/v1/auth/forgot-password	POST	Public
/api/v1/auth/reset-password/:token	PATCH	Public
/api/v1/users	GET	Admin
/api/v1/users/:id	GET	Admin
/api/v1/users/:id/status	PATCH	Admin
/api/v1/users/:id	DELETE	Admin
/api/v1/admin/stats	GET	Admin
/api/v1/profile	GET	Private
/api/v1/profile	PATCH	Private
/api/v1/profile/change-password	PATCH	Private
/api/v1/profile/extract-skills	POST	Job Seeker
/api/v1/jobs	GET	Public
/api/v1/jobs	POST	Recruiter (approved)
/api/v1/jobs/recommended	GET	Job Seeker
/api/v1/jobs/my-jobs	GET	Recruiter
/api/v1/jobs/saved	GET	Job Seeker
/api/v1/jobs/:id	GET	Public
/api/v1/jobs/:id	PATCH	Recruiter (owner)

/api/v1/jobs/:id	DELETE	Recruiter (owner) / Admin
/api/v1/jobs/:id/save	POST	Job Seeker
/api/v1/jobs/:jobId/apply	POST	Job Seeker
/api/v1/jobs/:jobId/applicants	GET	Recruiter (owner)
/api/v1/applications/my	GET	Job Seeker
/api/v1/applications/:id/status	PATCH	Recruiter
/api/v1/applications	GET	Admin

Business Rules

Your controllers must enforce all of the following. They will be tested:

- No duplicate applications.** A job seeker cannot apply to the same job twice. Return 400 if they try. Enforce this at the database level with a unique compound index on {user, job} in your Application model.
- Pending recruiters cannot post jobs.** A recruiter with `status: "pending"` must receive a 403 with an explanatory message. Only recruiters with `status: "approved"` may create, edit, or manage job posts.
- Recruiters own their posts.** A recruiter can only edit or delete jobs where `createdBy` matches `req.user._id`. Admins can delete any job post.
- Application status is gated by job ownership.** Only the recruiter who owns the job an application belongs to may update that application's status.
- Saved jobs must be open.** A job seeker can only bookmark jobs with `status: "open"`. Return 400 if they attempt to save a closed listing.

User Management (Admin Only)

GET /api/v1/users *Admin only. Returns a paginated list of users, filterable by role and status. Use `?role=recruiter&status=pending` to find recruiters awaiting approval.*

```

1 // Query params (all optional): ?role=recruiter&status=pending&page=1&limit=20
2 // Response 200
3 { "success": true, "total": 15, "page": 1,
4   "users": [ { "_id": "...", "name": "...", "email": "...",
5                 "role": "recruiter", "status": "pending",
6                 "createdAt": "..." }, ... ] }

```

GET /api/v1/users/:id *Admin only. Returns a single user by ID.*

```

1 // Response 200
2 { "success": true,
3   "user": { "_id": "...", "name": "...", "role": "...",
4             "status": "...", "email": "..." } }
5 // Error 404: { "success": false, "message": "User not found" }

```

PATCH /api/v1/users/:id/status *Admin only. Approves, rejects, or resets a user's status. The primary use case is approving pending recruiters.*

Field	Type	Required	Notes
status	String	Yes	approved rejected pending

```

1 // Response 200
2 { "success": true, "user": { "_id": "...", "status": "approved", ... } }

```

DELETE `/api/v1/users/:id` *Admin only. Permanently deletes a user account.*

```

1 // Response 200
2 { "success": true, "message": "User deleted" }
3 // Error 404: { "success": false, "message": "User not found" }

```

GET `/api/v1/admin/stats` *Admin only. Returns platform-wide statistics computed via MongoDB aggregation pipelines. Use `User.aggregate()`, `JobPost.aggregate()`, and `Application.aggregate()` with `$group` and `$sort` stages.*

```

1 // Response 200
2 {
3   "success": true,
4   "stats": {
5     "usersByRole": { "jobSeeker": 120, "recruiter": 34 },
6     "jobsByStatus": { "open": 58, "closed": 21 },
7     "appsByStatus": { "pending": 200, "shortlisted": 45, "rejected": 30 },
8     "topJobs": [
9       { "_id": "...", "title": "Backend Intern",
10         "company": "TechCo", "applicationCount": 18 }, ...
11     ]
12   }
13 }

```

Profile

All profile routes require a valid `Authorization: Bearer <token>` header.

GET `/api/v1/profile` *Private (any authenticated user). Returns the logged-in user's full profile.*

```

1 // Response 200
2 {
3   "success": true,
4   "user": { "_id": "...", "name": "...", "email": "...",
5             "bio": "...", "skills": [...], "profilePicture": "...",
6             "role": "...", "status": "..." }
7 }

```

PATCH `/api/v1/profile` *Private. Updates the logged-in user's own profile. All fields are optional.*

Field	Type	Required	Notes
name	String	No	Updated display name
bio	String	No	Used as input for skill extraction
profilePicture	String	No	URL to profile image

```

1 // Response 200
2 { "success": true, "user": { ... updated fields ... } }

```

PATCH `/api/v1/profile/change-password` *Private. Lets any authenticated user change their own password while logged in. This is separate from the forgot-password flow — it requires knowing the current password.*

Field	Type	Required	Notes
currentPassword	String	Yes	Must match the stored hash
newPassword	String	Yes	Min 6 characters

```

1 // Response 200
2 { "success": true, "message": "Password updated successfully" }
3 // Error 401: { "success": false, "message": "Current password is incorrect" }

```

POST /api/v1/profile/extract-skills *Job Seeker only. Reads the user's bio from the database and runs it through the HuggingFace NER model to extract skills. The cleaned results are saved back to user.skills.*

```

1 // No request body required
2 // Response 200
3 {
4   "success": true,
5   "skills": ["React", "Node.js", "MongoDB"], // saved to profile
6   "extracted": ["React", "Node.js", "MongoDB"] // raw NER output
7 }
8 // Error 400: { "success": false, "message": "Bio is empty. Update your profile first." }

```

Jobs

GET /api/v1/jobs *Public. Returns a paginated, filterable list of all job postings.*

```

1 // Query params (all optional):
2 // ?keyword=react&location=Cairo&type=internship&status=open&page=1&limit=10
3 // Response 200
4 {
5   "success": true, "total": 42, "page": 1,
6   "jobs": [ { "_id": "...", "title": "Backend Intern",
7               "company": "TechCo", "location": "Cairo",
8               "type": "internship", "category": "Backend",
9               "status": "open", "createdAt": "..." }, ... ]
10 }

```

POST /api/v1/jobs *Recruiter only (must have status: approved). Creates a new job post. The AI model automatically assigns category before the document is saved.*

Field	Type	Required	Notes
title	String	Yes	Job title
company	String	Yes	Company name
description	String	Yes	Used as input for AI classification
requirements	String[]	Yes	Array of required skills/technologies
location	String	Yes	City or "Remote"
type	String	Yes	"full-time" "part-time" "internship"
salary	Number	No	Monthly salary (optional)
totalSlots	Number	No	Open positions, defaults to 1

```

1 // Response 201
2 {
3   "success": true,
4   "job": { "_id": "...", "title": "Backend Intern",
5             "category": "Backend", // auto-assigned by AI
6             "status": "open", "createdBy": "<recruiter id>", ... }
7 }
8 // Error 403: { "success": false,
9 //   "message": "Your account is pending approval. Wait for admin approval
10 //   before posting jobs." }

```

GET /api/v1/jobs/recommended *Job Seeker only. Returns jobs ranked by cosine similarity between the user's skills and job requirements, computed via HuggingFace sentence embeddings.*

```
1 // No request body or query params needed
2 // Response 200
3 {
4   "success": true,
5   "jobs": [ { "_id": "...", "title": "...", "company": "...",
6               "category": "...", "score": 0.87, ... }, ... ]
7   // sorted by similarity score, highest first
8 }
```

GET /api/v1/jobs/my-jobs *Recruiter only. Returns all job posts created by the logged-in recruiter.*

```
1 // Response 200
2 { "success": true,
3   "jobs": [ { "_id": "...", "title": "...", "status": "open",
4               "type": "...", "createdAt": "...", ... } ] }
```

GET /api/v1/jobs/saved *Job Seeker only. Returns all jobs the logged-in user has saved/bookmarked.*

```
1 // Response 200
2 { "success": true,
3   "jobs": [ { "_id": "...", "title": "...", "company": "...",
4               "type": "...", "status": "open", ... } ] }
```

GET /api/v1/jobs/:id *Public. Returns a single job post by its ID, with recruiter info populated.*

```
1 // Response 200
2 { "success": true,
3   "job": { "_id": "...", "title": "...", "description": "...",
4             "requirements": [...], "category": "...", "status": "open",
5             "createdBy": { "name": "...", "email": "..." } } }
6 // Error 404: { "success": false, "message": "Job not found" }
```

PATCH /api/v1/jobs/:id *Recruiter only (must own the job). Updates a job post. If **description** is changed, the AI re-classifies the category automatically.*

```
1 // Request body (all fields optional)
2 { "title": "Senior Backend Engineer", "status": "closed" }
3 // Response 200
4 { "success": true, "job": { ... updated fields ... } }
5 // Error 403: { "success": false, "message": "Not authorised to edit this job" }
```

DELETE /api/v1/jobs/:id *Recruiter (owner) or Admin. Permanently deletes a job post.*

```
1 // Response 200
2 { "success": true, "message": "Job deleted" }
3 // Error 403: { "success": false, "message": "Not authorised to delete this job" }
```

POST /api/v1/jobs/:id/save *Job Seeker only. Toggles the saved state for a job. If already saved, it is removed; if not saved, it is added. Only open jobs can be saved.*

```
1 // No request body required
2 // Response 200 (job added to saved list)
3 { "success": true, "message": "Job saved", "saved": true }
4 // Response 200 (job removed from saved list)
5 { "success": true, "message": "Job removed from saved", "saved": false }
6 // Error 400: { "success": false, "message": "Cannot save a closed job" }
```

POST /api/v1/jobs/:jobId/apply *Job Seeker only. Submits an application to a job. Duplicate applications are rejected at the database level via a unique compound index.*

Field	Type	Required	Notes
coverLetter	String	No	Optional motivational text

```

1 // Response 201
2 {
3   "success": true,
4   "application": { "_id": "...", "user": "<userId>", "job": "<jobId>",
5                     "status": "pending", "appliedAt": "..." }
6 }
7 // Error 400: { "success": false, "message": "You have already applied to this job" }
8 // Error 404: { "success": false, "message": "Job not found" }

```

GET /api/v1/jobs/:jobId/applicants *Recruiter only (must own the job). Returns all applications for a specific job, with full applicant details populated.*

```

1 // Response 200
2 {
3   "success": true,
4   "applications": [
5     { "_id": "...", "status": "pending", "coverLetter": "...",
6       "appliedAt": "...",
7       "user": { "_id": "...", "name": "Sara Ahmed",
8                 "email": "...", "skills": ["React", "Node.js"] } }
9   ]
10 }

```

Applications

GET /api/v1/applications/my *Job Seeker only. Returns all applications submitted by the logged-in user, with job details populated.*

```

1 // Response 200
2 {
3   "success": true,
4   "applications": [
5     { "_id": "...", "status": "pending", "appliedAt": "...",
6       "job": { "_id": "...", "title": "Backend Intern",
7                "company": "TechCo", "type": "internship",
8                "status": "open" } }
9   ]
10 }

```

PATCH /api/v1/applications/:id/status *Recruiter only (must own the job the application belongs to). Updates the review status of an application.*

Field	Type	Required	Notes
status	String	Yes	"pending" "shortlisted" "rejected"

```

1 // Response 200
2 { "success": true,
3   "application": { "_id": "...", "status": "shortlisted", ... } }
4 // Error 403: { "success": false,
5 //             "message": "Not authorised to update this application" }

```

GET /api/v1/applications *Admin only. Returns a paginated list of all applications on the platform.*

```
1 // Query params: ?page=1&limit=20
2 // Response 200
3 { "success": true, "total": 120, "page": 1,
4   "applications": [ { "_id": "...",
5                       "user": { "_id": "...", "name": "...", "email": "...", },
6                       "job": { "_id": "...", "title": "...", "company": "...", },
7                       "status": "...", "appliedAt": "..." }, ... ] }
```

5 Hugging Face API Integration

This is what separates GIU Nexus from a plain CRUD app. Three AI models work behind the scenes to extract skills, classify jobs, and recommend listings — all triggered automatically by your existing endpoints. **Every Hugging Face call must originate from your Express backend, never from the React frontend.** Keeping your HF_TOKEN server-side is not optional.

Client Initialisation

Create `services/hfService.js` and initialise the client once, then export the singleton:

```
1 const { HfInference } = require("@huggingface/inference");
2 const hf = new HfInference(process.env.HF_TOKEN);
3 module.exports = hf;
```

Import this singleton in your controllers. Do **not** call `new HfInference()` multiple times — instantiate once, reuse everywhere.

1. Skill Extraction via NER — POST /api/v1/profile/extract-skills

Model: `dslim/bert-base-NER`

When a job seeker calls this endpoint, your backend reads their `bio` from the database and sends it to the NER model. The model returns tagged entities — filter for `B-MISC`, `I-MISC`, and `B-ORG` tags (technologies and organisations), deduplicate, and save the clean array to `user.skills`.

```
1 const result = await hf.tokenClassification({
2   model: "dslim/bert-base-NER",
3   inputs: user.bio,
4 });
5 // result: [{ word, entity_group, score, ... }, ...]
6 // Filter by entity_group, map to word strings, deduplicate, save:
7 user.skills = cleanSkills;
8 await user.save();
```

Graceful failure: If the call throws, catch the error, log it, and return the user's *existing* skills unchanged with 200.

2. Job Recommendation via Embeddings — GET /api/v1/jobs/recommended

Model: `sentence-transformers/all-MiniLM-L6-v2`

- Build a text string from the logged-in job seeker's `skills` array (join with commas).
- Fetch all `JobPost` documents with `status: open` and build a summary string for each (e.g., join the job title and requirements array into a single string).
- Send the student text and all job texts to `featureExtraction` in a single call.
- Compute **cosine similarity** between the student vector (index 0) and each job vector.

e) Sort jobs by score descending and return.

```
1 const embeddings = await hf.featureExtraction({
2   model: "sentence-transformers/all-MiniLM-L6-v2",
3   inputs: [studentText, ...jobTexts],
4 });
5
6 function cosineSimilarity(vecA, vecB) {
7   const dot = vecA.reduce((sum, a, i) => sum + a * vecB[i], 0);
8   const magA = Math.sqrt(vecA.reduce((sum, a) => sum + a * a, 0));
9   const magB = Math.sqrt(vecB.reduce((sum, b) => sum + b * b, 0));
10  return dot / (magA * magB);
11 }
```

Graceful failure: If the embedding call fails, fall back to returning all open jobs without scores.

3. Job Category Classification — triggered on POST /api/v1/jobs

Model: facebook/bart-large-mnli

Before saving a new job post, automatically classify its description using zero-shot classification. Store the top-scoring label in `jobPost.category`. No extra endpoint is needed — this runs inside `createJob`.

```
1 const result = await hf.zeroShotClassification({
2   model: "facebook/bart-large-mnli",
3   inputs: [req.body.description],
4   parameters: { candidate_labels: ["Frontend", "Backend", "AI/ML",
5                                     "DevOps", "Data Engineering", "Other"] },
6 });
7 const category = result[0].labels[0]; // highest-scoring label
```

Graceful failure: If classification fails, default to `category: "Other"` and continue saving the job normally.

6 General Error Handling

Your API must behave predictably under every failure condition.

a) **Centralised error middleware.** Register it as the *last* `app.use()` call in `server.js`. Any controller that calls `next(err)` will land here. Always respond with:

```
1 { "success": false, "message": "<human-readable message>" }
```

b) **Consistent HTTP status codes.** Use the right code every time — your clients depend on it.

Status	When to use
200	Successful GET, PATCH, DELETE
201	Successful POST (resource created)
400	Validation error, bad request body, duplicate data
401	Missing or invalid JWT
403	Authenticated but not authorised (wrong role or not owner)
404	Resource not found
500	Unexpected server error

7 Bonus Features

The following features go beyond the base requirements and each earn bonus marks. They are grouped by effort level — pick the ones that interest your team. You are not required to implement all of them.

Feature	What to build	Effort
Rate Limiting	Install <code>express-rate-limit</code> and apply it to auth routes (<code>/login</code> , <code>/register</code> , <code>/forgot-password</code>). Limit to e.g., 10 requests per 15-minute window per IP. Return 429 when exceeded.	Low
Token Blacklist Logout	Maintain an in-memory Set (or Redis cache) of invalidated JWT IDs. When a user logs out, add the token's <code>jti</code> claim to the blacklist. The <code>protect</code> middleware rejects blacklisted tokens immediately.	Medium
OTP-based MFA	In <code>/forgot-password</code> , generate a 6-digit OTP and send it via email before issuing the reset link. Add a <code>POST /api/v1/auth/verify-otp</code> step that must be completed first. Store the OTP with a short expiry on the user document.	Medium
Swagger / OpenAPI Docs	Add <code>swagger-jsdoc</code> and <code>swagger-ui-express</code> . Annotate your route files with JSDoc comments following the OpenAPI 3.0 spec. Serve the interactive docs at <code>GET /api-docs</code> .	Medium
Profile Picture Upload	Integrate <code>multer</code> for multipart form-data parsing and <code>Cloudinary</code> (or local disk storage) for the actual file. Update <code>PATCH /api/v1/profile</code> to accept a file in addition to JSON fields.	Medium
Extended Admin Stats	Extend <code>GET /api/v1/admin/stats</code> to include time-series data (e.g., applications per week over the last 4 weeks) using <code>\$dateToString</code> in your aggregation pipeline.	Medium
Jest + Supertest Tests	Write integration tests covering: register, login, create job (with AI category), apply to job, extract skills, and duplicate-application rejection. Use <code>mongodb-memory-server</code> so tests are self-contained and run without a real database.	High
CI/CD via GitHub Actions	Create <code>.github/workflows/ci.yml</code> that triggers on every push to <code>main</code> : install dependencies, run <code>npm test</code> , and optionally deploy to Render or Railway.	High
Docker Containerisation	Write a <code>Dockerfile</code> for your server and a <code>docker-compose.yml</code> that spins up both the server and a MongoDB container. The entire backend should start with a single <code>docker compose up</code> command.	High

8 Testing Your API

Before submission, verify that every endpoint works end-to-end using **Postman** or **Thunder Client**. Here is the recommended workflow.

Postman Environment Setup

Create a Postman environment so you never have to paste tokens manually:

Variable	Value
BASE_URL	http://localhost:5000/api/v1
TOKEN	set automatically after login (see below)
JOB_ID	set manually after creating a job
APP_ID	set manually after applying

In your login request's **Tests** tab, add the following snippet to capture the token automatically:

```
1 const res = pm.response.json();  
2 if (res.token) pm.environment.set("TOKEN", res.token);
```

Set the Authorization header on protected requests to: **Bearer** TOKEN.

Required Test Scenarios

You must be able to demonstrate all of the following:

- Full auth flow:** Register as a job seeker → login → receive token → use it on a protected route.
- Recruiter approval:** Register as a recruiter → confirm **status**: "pending" → log in as admin → approve recruiter → recruiter can now create jobs.
- AI category assignment:** Recruiter creates a job → response includes **category** auto-assigned by the AI model.
- Skill extraction:** Job seeker updates **bio** → calls **POST /profile/extract-skills** → skills appear on their profile.
- Job recommendations:** Job seeker with skills calls **GET /jobs/recommended** → response is a ranked list with **score** values.
- Full application flow:** Job seeker applies → recruiter sees applicant → recruiter shortlists → job seeker's **GET /applications/my** reflects the updated status.
- Business rule enforcement:** Apply to the same job twice → expect 400. Post a job as a pending recruiter → expect 403.

9 Team Organisation & Workflow

Dividing the Work

Your team should distribute the API endpoints in the API Routes table above equally among all members. Each member is responsible for **2–3 endpoints**, including the controller logic, route wiring, and a Postman test for each endpoint they own.

Team Lead

Before writing a single line of code, designate one member as the **Team Lead**. The team lead's responsibilities are:

- Review every Pull Request (PR) before it is merged into **main**.
- Verify that the feature exactly matches the requirements in this milestone document.

- Be the only person who approves and performs (or explicitly authorises) the final merge to **main**.
- Reject PRs that break existing functionality or deviate from the specification.

No member may merge their own feature branch to main without the team lead's written approval on the PR. Unreviewed merges that break the stable branch will count against your submission.

Branching Strategy

- Create a feature branch for every endpoint or closely related group of endpoints, e.g. **feature/auth-login**, **feature/job-recommendations**.
- Work exclusively on your own branch — never directly on **main**.
- When done, open a PR against **main**, assign it to the team lead, and include a Postman screenshot as evidence that the endpoint works.
- The team lead reviews, requests changes if needed, then approves and merges.

Technology Stack — Non-Negotiable

You must use the **MERN stack** and nothing else:

Layer	Required Technology
Database	MongoDB (via Mongoose)
Backend	Express.js on Node.js
Frontend	React.js

Substituting MongoDB with another database, Express with another framework, or Node.js with another runtime is **not permitted**. Doing so constitutes a violation of the assignment requirements and **academic misconduct under GIU university policy**, which carries zero tolerance. Similarly, do not add any library or feature that is not listed in this milestone's requirements — unless it appears in the **Bonus Features section** (Section 7).

Bonus Ranking & Final Presentation

Teams that successfully implement one or more bonus features will be ranked as **top-performing teams** at the end of the semester. The top teams will be invited to give a **live presentation** in front of an audience of students, instructors, and guests, where they demonstrate and explain what they built. This is your chance to go beyond the standard requirements and showcase your engineering skills.

10 Project Management Tools

A dedicated project management tool keeps your team aligned, ensures no endpoint is forgotten or duplicated, and gives the team lead a clear picture of who is working on what. We recommend **Jira** (free for students via atlassian.com/software/jira/free) or **Asana** (app.asana.com).

Recommended Workflow

- Create a project board.** In Jira, choose the *Scrum* template; in Asana, choose *Board*. A single two-week sprint maps neatly to the Milestone 2 deadline.
- Create one ticket per endpoint** (or closely related group). Use a clear naming convention:

[POST] /api/v1/auth/register [GET] /api/v1/jobs/recommended

- c) **Assign each ticket** to the team member responsible for that endpoint.
- d) **Use columns to reflect the real status of the work:**
Backlog → In Progress → In Review (PR open) → Done (merged & tested).
- e) **Link your GitHub PR to the Jira/Asana ticket.** This gives the team lead one place to see the code and the ticket status together.
- f) **Hold a brief daily stand-up** (10 minutes maximum): what you completed yesterday, what you will do today, and any blockers.
- g) **Close the ticket only after the PR is merged** and the endpoint has been tested end-to-end in Postman.

Further Reading

For a visual walkthrough of setting up a Jira Scrum board and running a sprint (official Atlassian guide):
<https://www.atlassian.com/agile/scrum/sprint-planning>

For sprint-specific best practices covering backlog grooming, velocity, and daily stand-ups:
<https://community.atlassian.com/forums/Jira-articles/Sprint-Best-Practices/ba-p/2900429>

11 Milestone 2 Deadline

Milestone 2 deadline is **Friday, 8th May 2026, 11:59 pm**

12 Submission

- a) Push all Milestone 2 work to your GitHub repository via Pull Requests from feature branches. Each major feature should live on its own branch.
- b) Submit your repository link (if changed) and a brief video or screenshots demonstrating the three AI-powered endpoints through the following form:
<https://forms.gle/5PYtDDysWXntAAGp7>
- c) **Do not commit your .env file.** Your `.gitignore` will be checked. Include a `.env.example` with placeholder values so the grader can see what variables are required without exposing any secrets.